Digital System Design

Dr. Nazanin Ghasemian

Fall 2026 · Lecture 3

Last Lecture

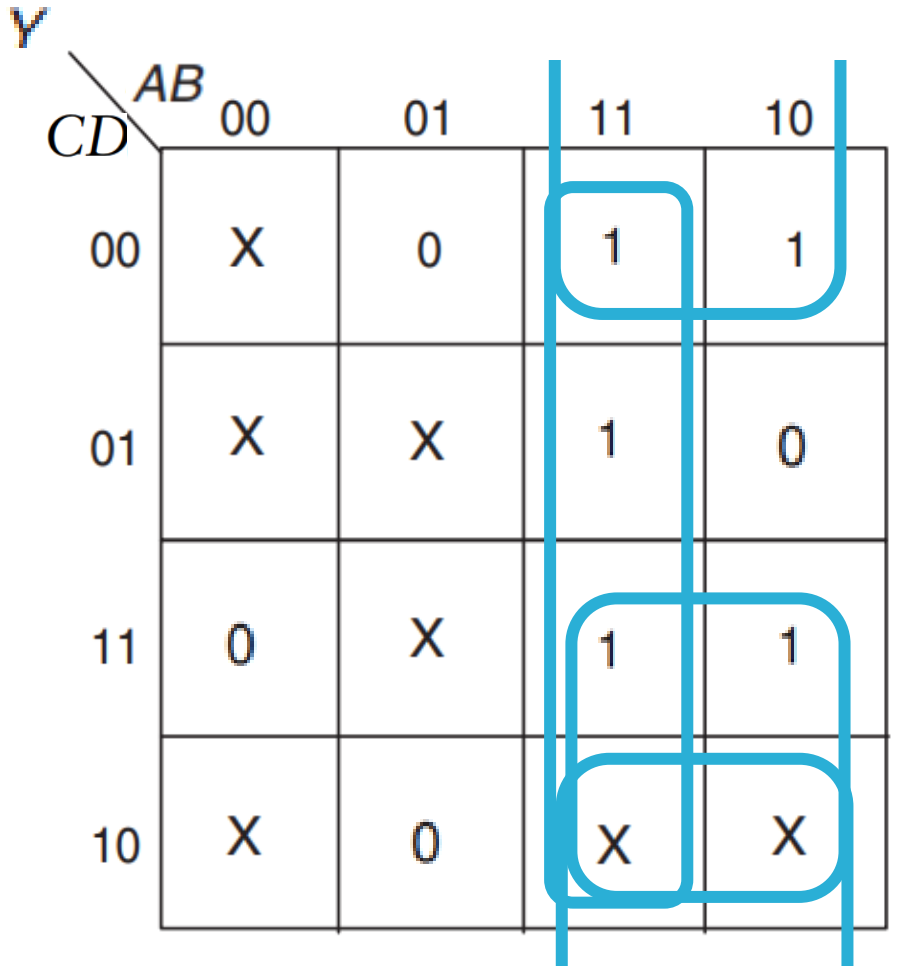
- Reviewed logic gates
- Reviewed combinational logic
- Reviewed basic important combinational circuits

Example 1

Find a minimal Boolean equation for the function in the truth table. Remember to take advantage of the don’t care entries.

A	B	C	D	Y
0	0	0	0	X
0	0	0	1	X
0	0	1	0	X
0	0	1	1	0
0	1	0	0	0
0	1	0	1	X
0	1	1	0	0
0	1	1	1	X
1	0	0	0	1
1	0	0	1	0
1	0	1	0	X
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	X
1	1	1	1	1

Solution



$A\overline{D}$

AC

AB

$Y = A\overline{D} + \mathrm{AC} + AB$

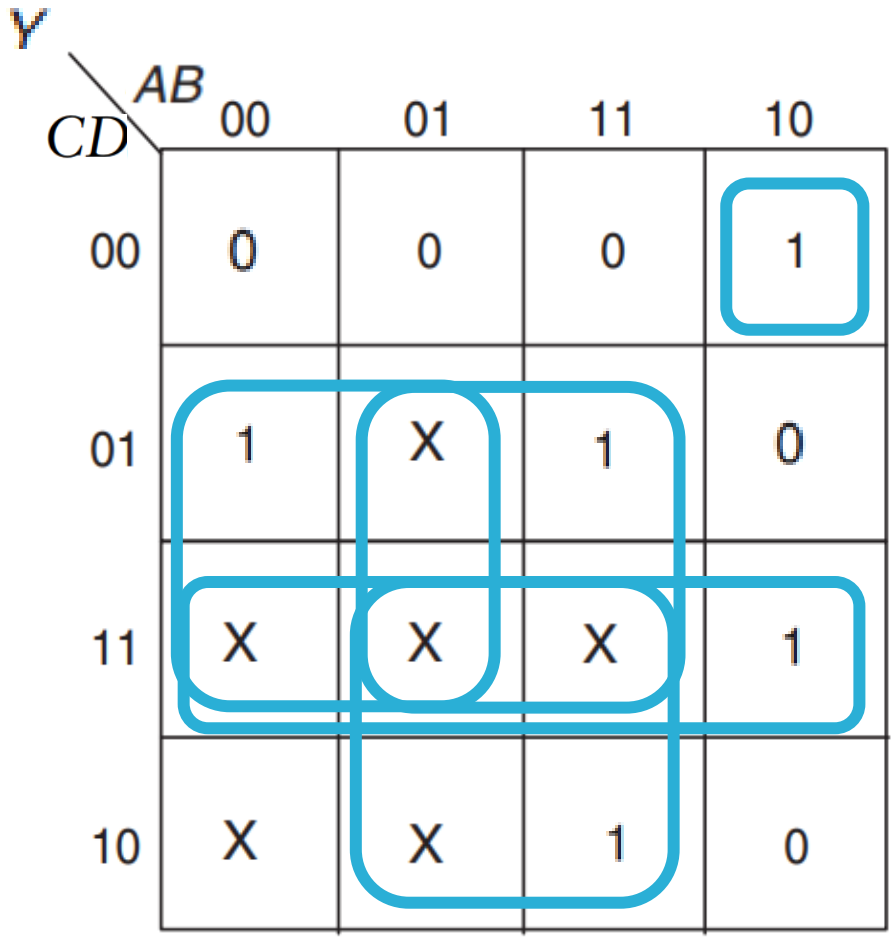
A	B	C	D	Y
0	0	0	0	X
0	0	0	1	X
0	0	1	0	X
0	0	1	1	0
0	1	0	0	0
0	1	0	1	X
0	1	1	0	0
0	1	1	1	X
1	0	0	0	1
1	0	0	1	0
1	0	1	0	X
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	X
1	1	1	1	1

Quiz

Find a minimal Boolean equation for the function in the truth table. Remember to take advantage of the don’t care entries.

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	X
0	0	1	1	X
0	1	0	0	0
0	1	0	1	X
0	1	1	0	X
0	1	1	1	X
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	X
1	1	1	1	1

Quiz - Solution



A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	X
0	0	1	1	X
0	1	0	0	0
0	1	0	1	X
0	1	1	0	X
0	1	1	1	X
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	X
1	1	1	1	1

BD

$\bar{A}D$

BC

CD

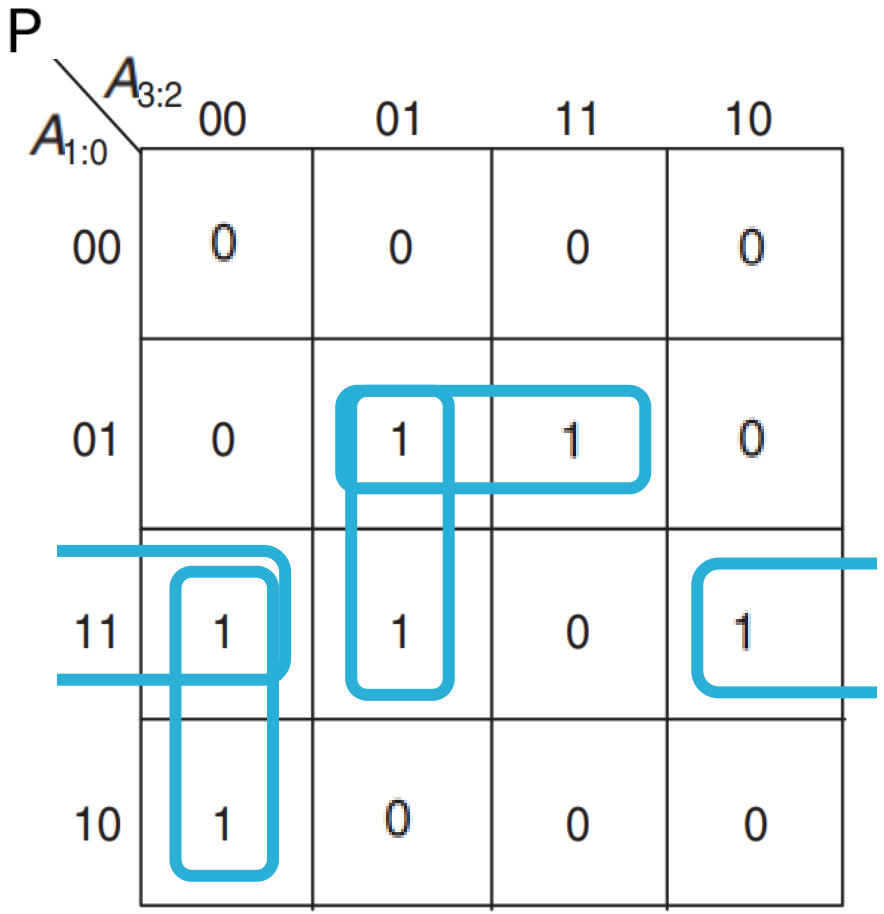
$A\bar{B}\,\bar{C}\,\bar{D}$

$$Y = BD + \bar{A}D + BC + CD + A\bar{B}\,\bar{C}\,\bar{D}$$

Example 2

A circuit has four inputs and two outputs. The inputs A3:0 represent a number from 0 to 15. Output P should be TRUE if the number is prime (0 and 1 are not prime, but 2, 3, 5, and so on, are prime). Output D should be TRUE if the number is divisible by 3. Give simplified Boolean equations for each output and sketch a circuit.

Solution



$A_2\overline{A_1}A_0$

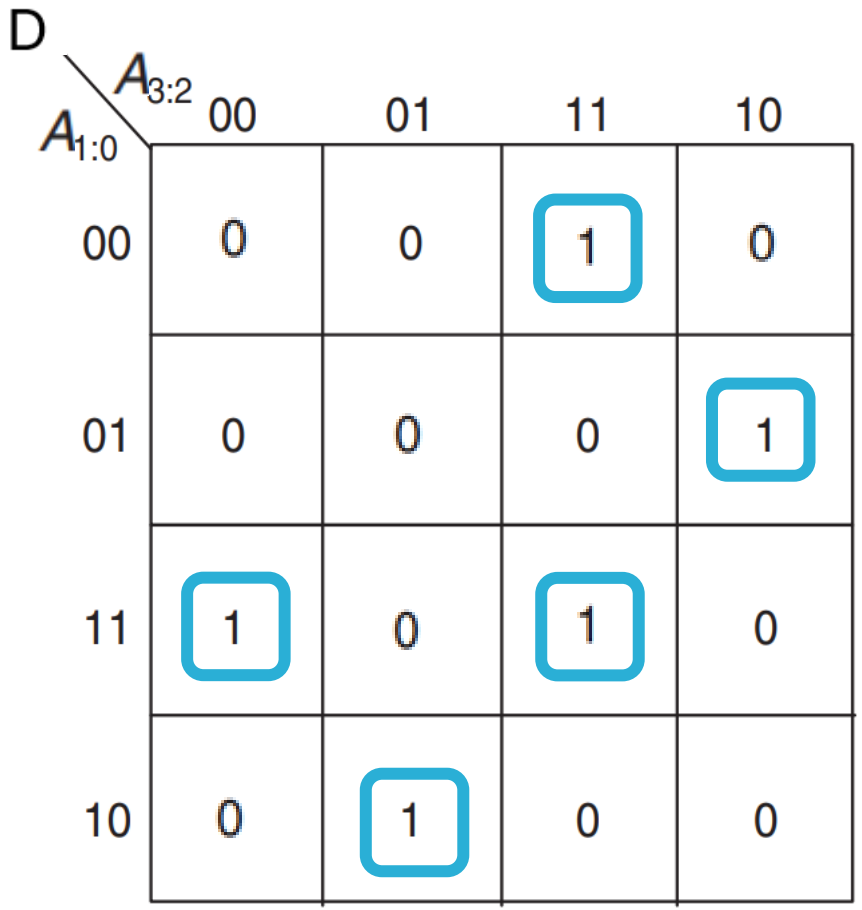
$\overline{A_3}A_2A_0$

$\overline{A_3}\,\overline{A_2}A_1$

$\overline{A_2}A_1A_0$

A3	A2	A1	A0	P	D
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	1	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	1	0	1
1	0	1	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	1	0	1

Solution



$A_3 A_2 \overline{A_1}\,\overline{A_0}$

$A_3 \overline{A_2}\,\overline{A_1} A_0$

$\overline{A_3}\,\overline{A_2} A_1 A_0$

$A_3 A_2 A_1 A_0$

$\overline{A_3} A_2 A_1 \overline{A_0}$

A3	A2	A1	A0	P	D
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	1	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	1	0	1
1	0	1	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	1	0	1

Hardware Description Language (HDL)

- Most commercial designs built using HDLs
- Two leading HDLs:
  - **SystemVerilog**
    - Developed in 1984 by Gateway Design Automation
    - IEEE standard (1364) in 1995
    - Extended in 2005 (IEEE STD 1800-2009)
  - **VHDL 2008**
    - Developed in 1981 by the Department of Defense
    - IEEE standard (1076) in 1987
    - Updated in 2008 (IEEE STD 1076-2008)

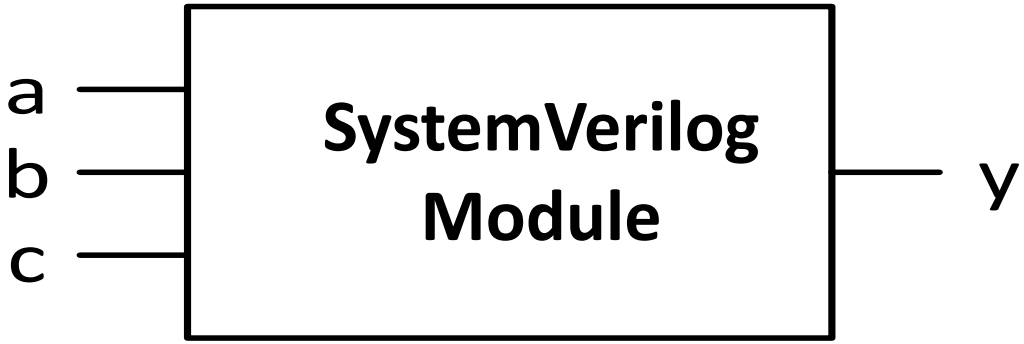
Hardware Description Language (HDL)

- **Simulation**
  - Inputs applied to circuit
  - Outputs checked for correctness
  - Millions of dollars saved by debugging in simulation instead of hardware
- **Synthesis**
  - Transforms HDL code into a *netlist* describing the hardware (i.e., a list of gates and the wires connecting them)

SystemVerilog Modules

Two types of Modules:

- **Behavioral:** describe what a module does
- **Structural:** describe how it is built from simpler modules



SystemVerilog Modules

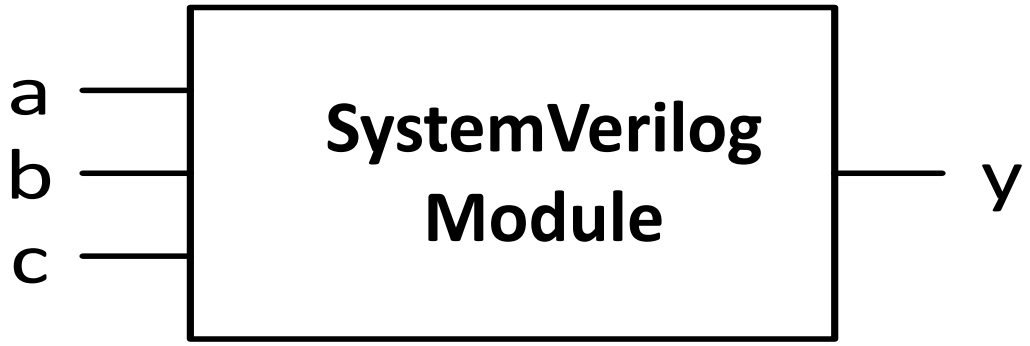
A SystemVerilog module begins with the module name and a listing of the inputs and outputs.

Module Declaration

A SystemVerilog module begins with the module name and a listing of the inputs and outputs.

```
module example(input  logic a, b, c,
               output logic y);
  // module body goes here
endmodule
```

- **`module`/`endmodule`:** required to begin/end module
- **`example`:** name of the module

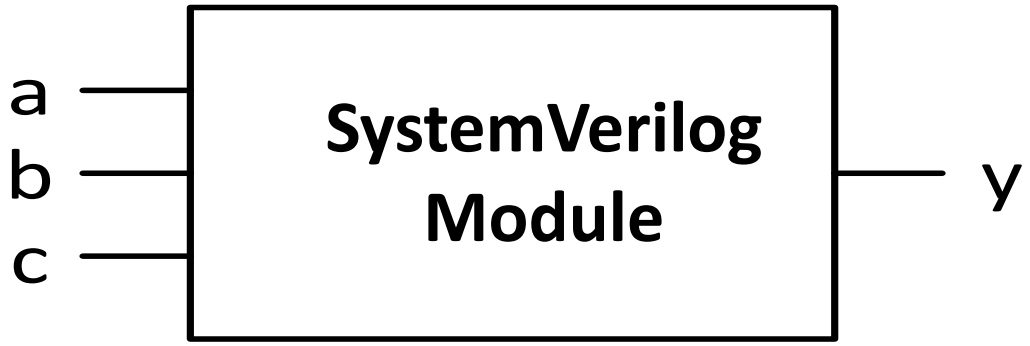


Module Declaration

A SystemVerilog module begins with the module name and a listing of the inputs and outputs.

```
module example(input  logic a, b, c,
               output logic y);
  // module body goes here
endmodule
```

`logic` signals are Boolean variables (0 or 1). They may also have floating (z) and undefined (x) values.



Solution – Hardware Description Language

- We want to describe this hardware in a way that is understandable to the simulation hardware.
- For that, we use HDL

Solution – Hardware Description Language

Module name

```
module Prime_Multiple3(
    input  logic [3:0] A,    // 4-bit input
    output logic       P,    // 1-bit output  prime
    output logic       D     // 1-bit output  multiple of 3
    );
endmodule
```

All ports with their types

Module

Solution – Hardware Description Language

Module name

```
module Prime_Multiple3(A, P, D);
    input  logic [3:0] A;   // 4-bit input
    output logic       P;   // 1-bit output  prime
    output logic       D;   // 1-bit output  multiple of 3
endmodule
```

All ports with their types

Module

Behavioral SystemVerilog

```
module example(input  logic a, b, c,
               output logic y);
  assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b &  c;
Endmodule
```

The **`assign`** statement describes combinational logic.

`~` indicates **`NOT`**,

`&` indicates **`AND`**,

and **`|`** indicates **`OR`**

Behavioral SystemVerilog

```
module Prime_Multiple3(
    input  logic [3:0] A,    // 4-bit input
    output logic       P,    // 1-bit output  prime
    output logic       D     // 1-bit output  multiple of 3
    );

assign P = (A[2] & ~A[1] & A[0])|(~A[3] & A[2] & A[0])|(~A[3] & ~A[2] & A[1])|(~A[2] & A[1] & A[0]);

endmodule
```

$$P = A_2\overline{A_1}A_0 + \overline{A_3}A_2A_0 + \overline{A_3}\,\overline{A_2}A_1 + \overline{A_2}A_1A_0$$

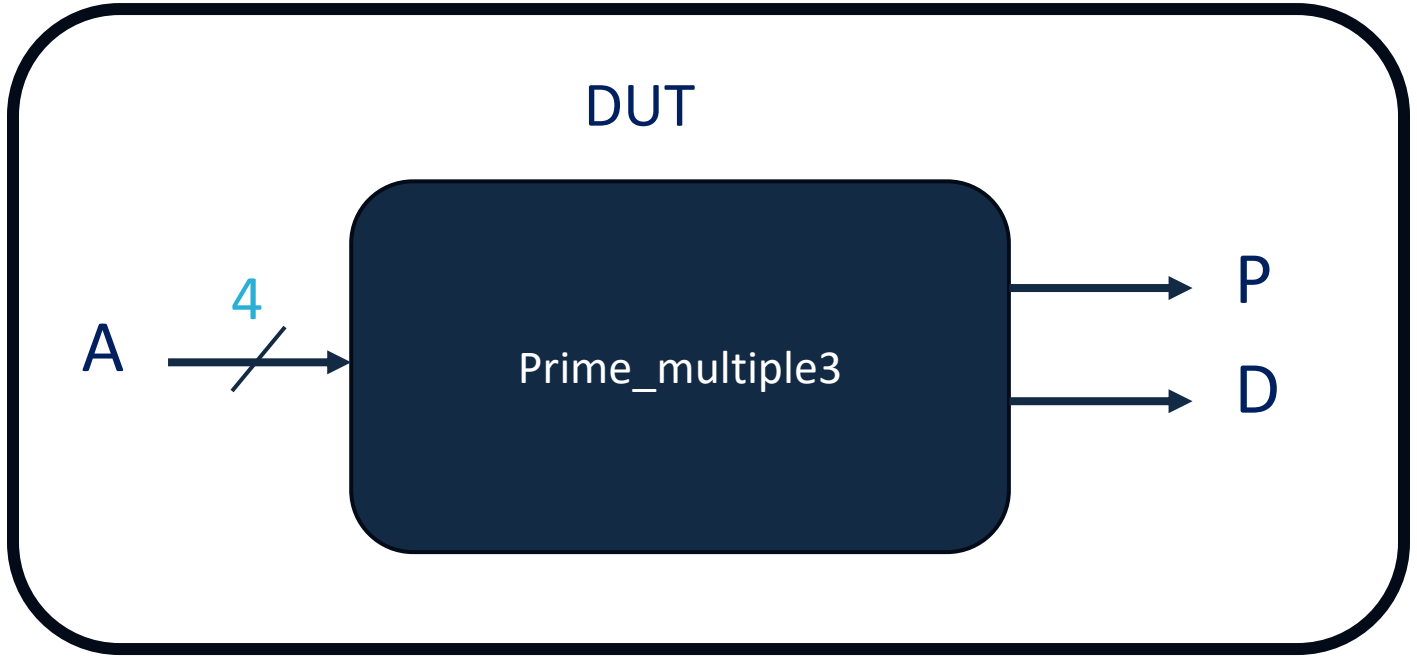
Testing Purpose

Use HDL to simulate hardware before physical implementation

1. Define the hardware using HDL constructs
2. Verify functionality with test cases (manual or automated)
   - For larger designs, employ ***Testbenches*** for efficient testing
   - What is a Testbench ?

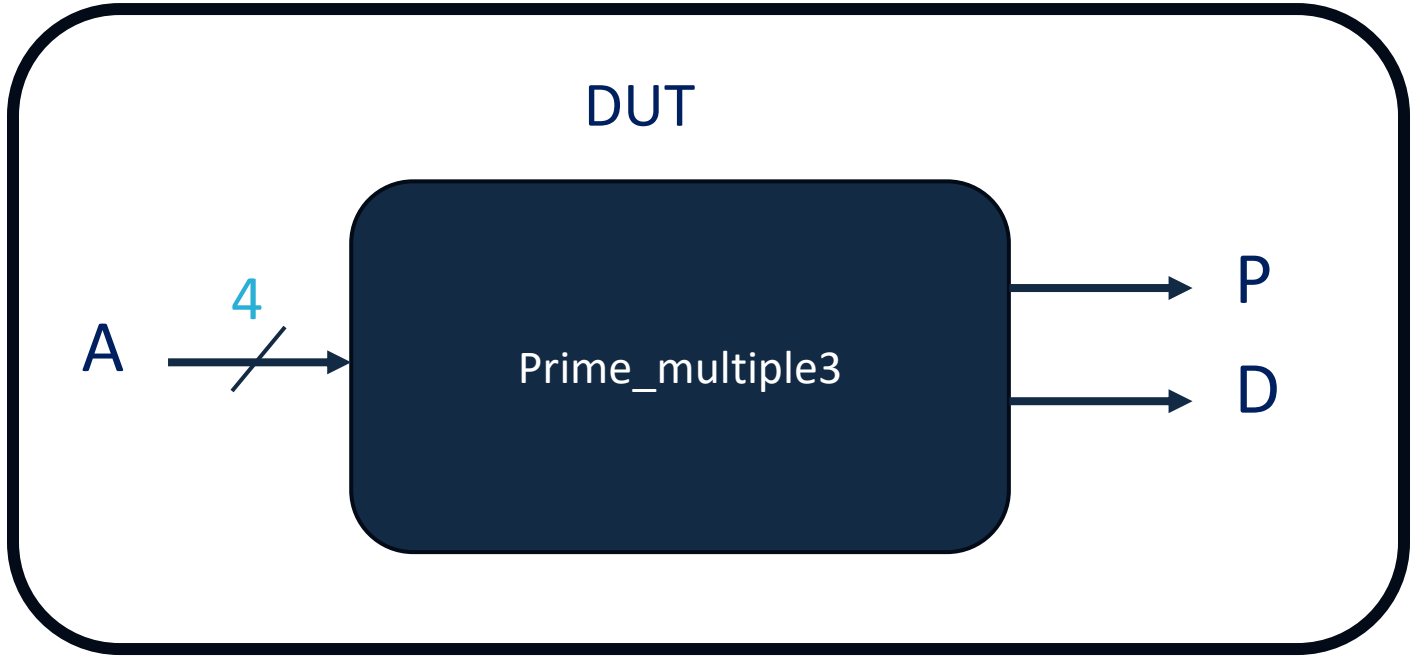
Testbench

A **`testbench`** is an HDL module that is used to test another module, called the **`device under test`** (*DUT*).

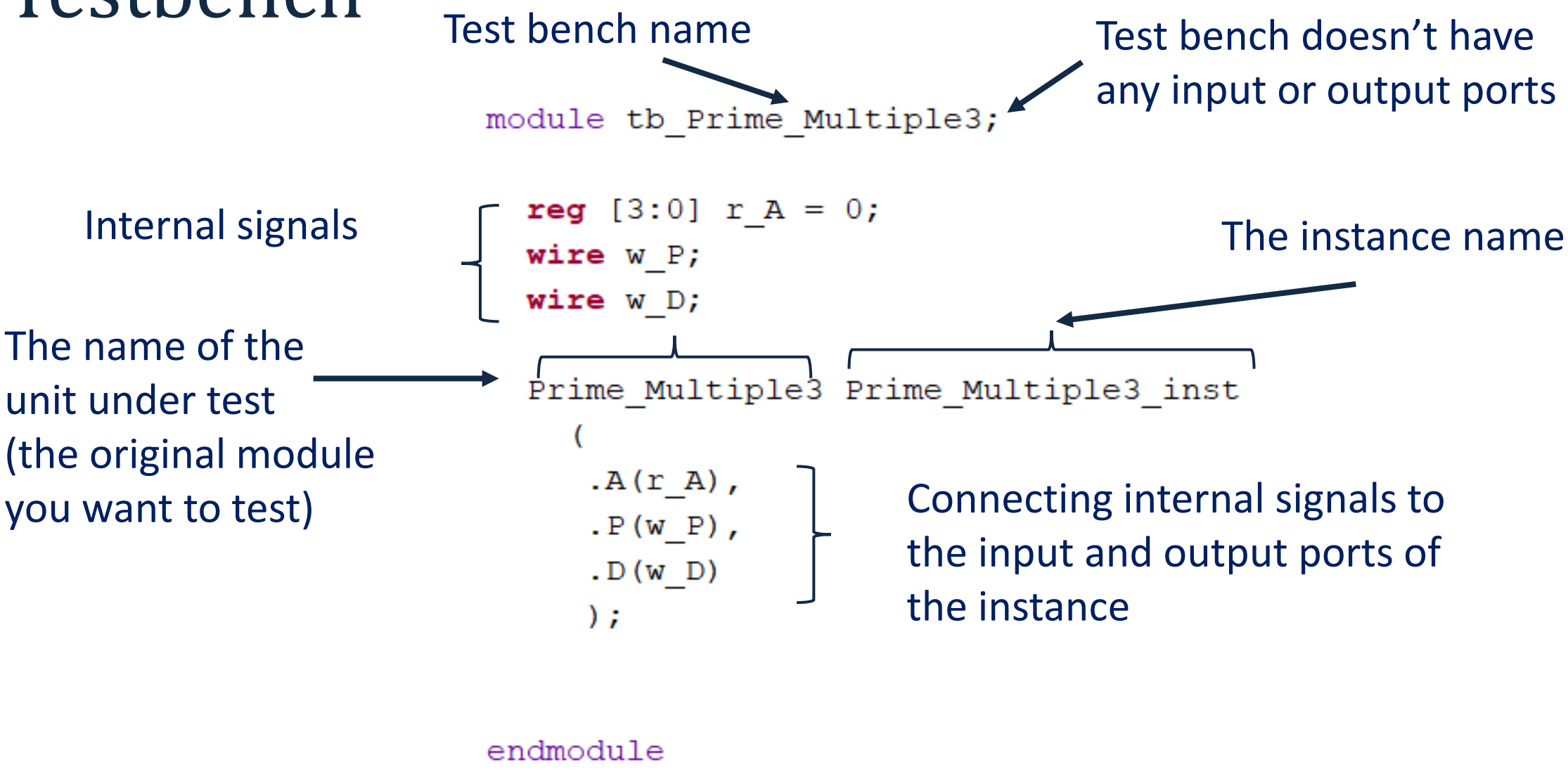


Testbench

`Testbench` contains statements to apply inputs to the DUT and, ideally, to check that the correct outputs are produced.

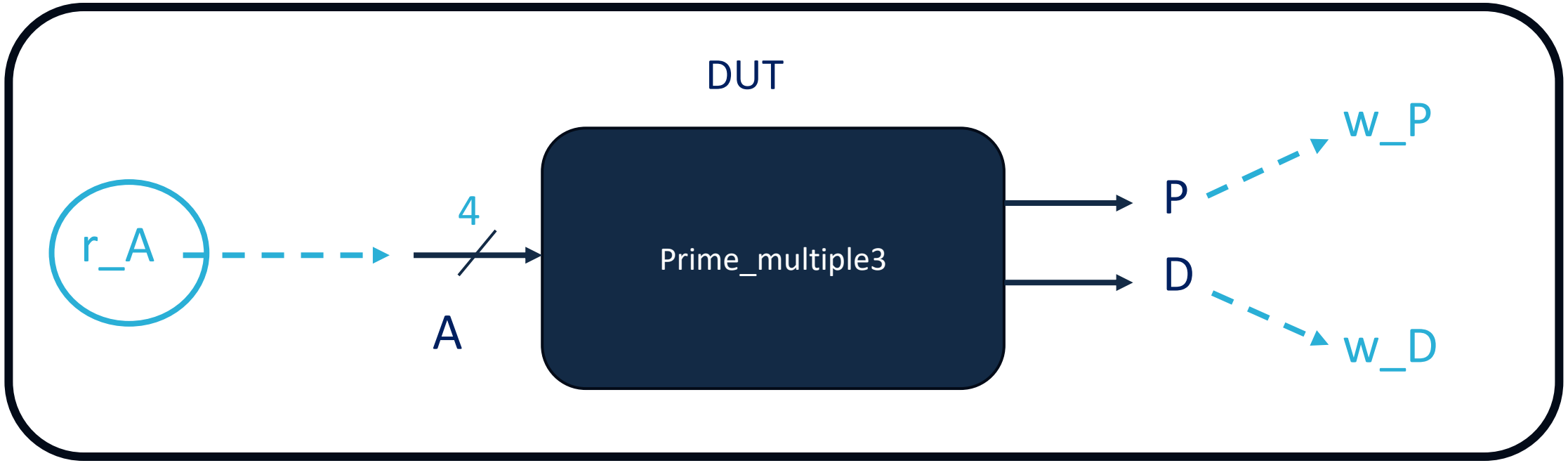


Testbench


Test bench name
Test bench doesn't have any input or output ports
module tb_Prime_Multiple3;
Internal signals
reg [3:0] r_A = 0;
wire w_P;
wire w_D;
The instance name
The name of the unit under test (the original module you want to test)
Prime_Multiple3 Prime_Multiple3_inst
(
.A(r_A),
.P(w_P),
.D(w_D)
);
Connecting internal signals to the input and output ports of the instance
endmodule

Testbench

Testbench



Provide test values for r_A

Observe the values in w_P and w_D

Testbench

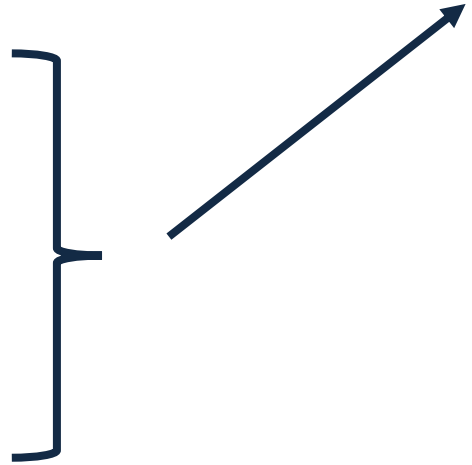
```
module tb_Prime_Multiple3;

  reg [3:0] r_A = 0;
  wire w_P;
  wire w_D;

  Prime_Multiple3 Prime_Multiple3_inst
    (
     .A(r_A),
     .P(w_P),
     .D(w_D)
     );

  initial
    begin
        r_A = 4'b0000;
        #10;
        r_A = 4'b0001;
        #10;
        r_A = 4'b1111;
        #10;
        r_A = 4'b1101;
        #10;
    end

endmodule
```



```
initial
  begin
      r_A = 4'b0000;
      #10;
      r_A = 4'b0001;
      #10;
      r_A = 4'b1111;
      #10;
      r_A = 4'b1101;
      #10;
  end
```

4 bit binary with value 0

10 ns wait